

Diagrama de clases — SGR

Documento generado desde `clases.md` del repositorio

Criterio 5 de la rúbrica — 15 puntos. 10 de septiembre de 2026 · Equipo Origami SpA

La estructura lógica del software: las **22 clases del dominio** con sus atributos tipados y sus multiplicidades, y los **12 servicios** que cargan las operaciones. Los nombres son los mismos del DER y de los casos de uso, porque la rúbrica evalúa esa coherencia como criterio transversal.

1. Qué pide la rúbrica, y dónde está

Lo que pide la rúbrica §6.1	Dónde está
Clases del dominio y/o componentes relevantes	Las 22 entidades en §4, §5 y §6; los 12 servicios en §7
Atributos significativos con tipo de dato	Todos, con tipo. La equivalencia con MySQL está en §9
Métodos u operaciones relacionados con las responsabilidades de la clase	Cada entidad lleva las suyas; la capa de servicios lleva las transversales, con los nombres reales del código
Visibilidad (+, -, #) cuando se use UML formal	En los cinco diagramas. + público, - privado, # protegido
Asociaciones con multiplicidades	En todas las asociaciones: 1, 0..1, 0..*, 1..*
Herencia, composición o agregación solo cuando exista justificación	Composición donde la parte no vive sin el todo; agregación donde sí. Herencia: ninguna , y §8 explica por qué
Nombres coherentes con los casos de uso y con el modelo de datos	La tabla de §10 cruza clase ↔ tabla ↔ caso de uso

2. De dónde sale el diagrama, y por qué de ahí

De dos fuentes, porque ninguna sola cumple la rúbrica.

- `backend/prisma/schema.prisma` da la estructura: las 22 entidades, sus atributos, sus tipos y sus multiplicidades. Es también la fuente del DER, y usar la misma garantiza que los nombres coincidan. Lo que **no** tiene es una sola operación: es un esquema de persistencia.
- `backend/src/services/` y `backend/src/routes/` dan el comportamiento. Ahí están las operaciones con su nombre real: `calcularPeriodo()`, `detectarDuplicidad()`, `actualizarConVersion()`, `generarCodigo()`.

Dibujar solo lo primero deja un diagrama sin métodos, que es literalmente uno de los seis puntos evaluados. Dibujar solo lo segundo deja un diagrama sin atributos ni multiplicidades, que son otros dos. Van los dos, y el §3 dice cuál mirar para qué.

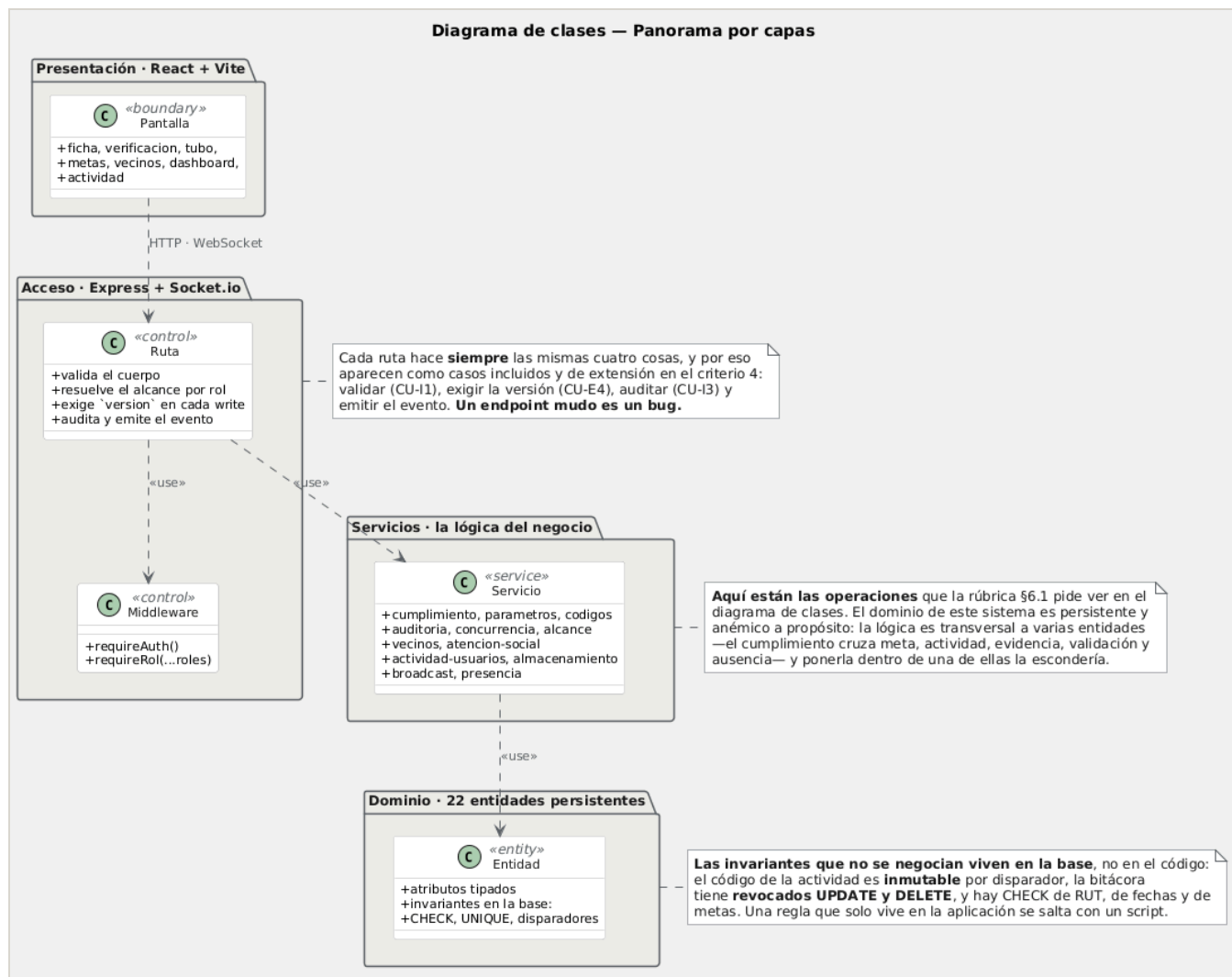
Ninguna operación de estos diagramas está inventada. Si no existe en el código, no está dibujada. Donde una clase tiene la tabla pero no el comportamiento, va con **borde punteado** y se dice cuál RF queda pendiente — la misma convención del diagrama de requerimientos.

3. Los cinco diagramas

#	Diagrama	Qué responde
1	<u>Panorama por capas</u>	Dónde vive cada cosa, y por qué el dominio no carga la lógica
2	<u>Dominio de la medición</u>	Cómo se llega desde el cargo hasta el puntaje
3	<u>Agenda colectiva y personas</u>	El tubo, el vecino y el caso social
4	<u>Plataforma y trazabilidad</u>	Multi-tenant, roles y bitácora
5	<u>Servicios</u>	Las operaciones del sistema

Van separados por legibilidad, que la rúbrica exige explícitamente: veintidós clases con atributos y métodos en un solo lienzo no se leen impresas.

4. Panorama por capas

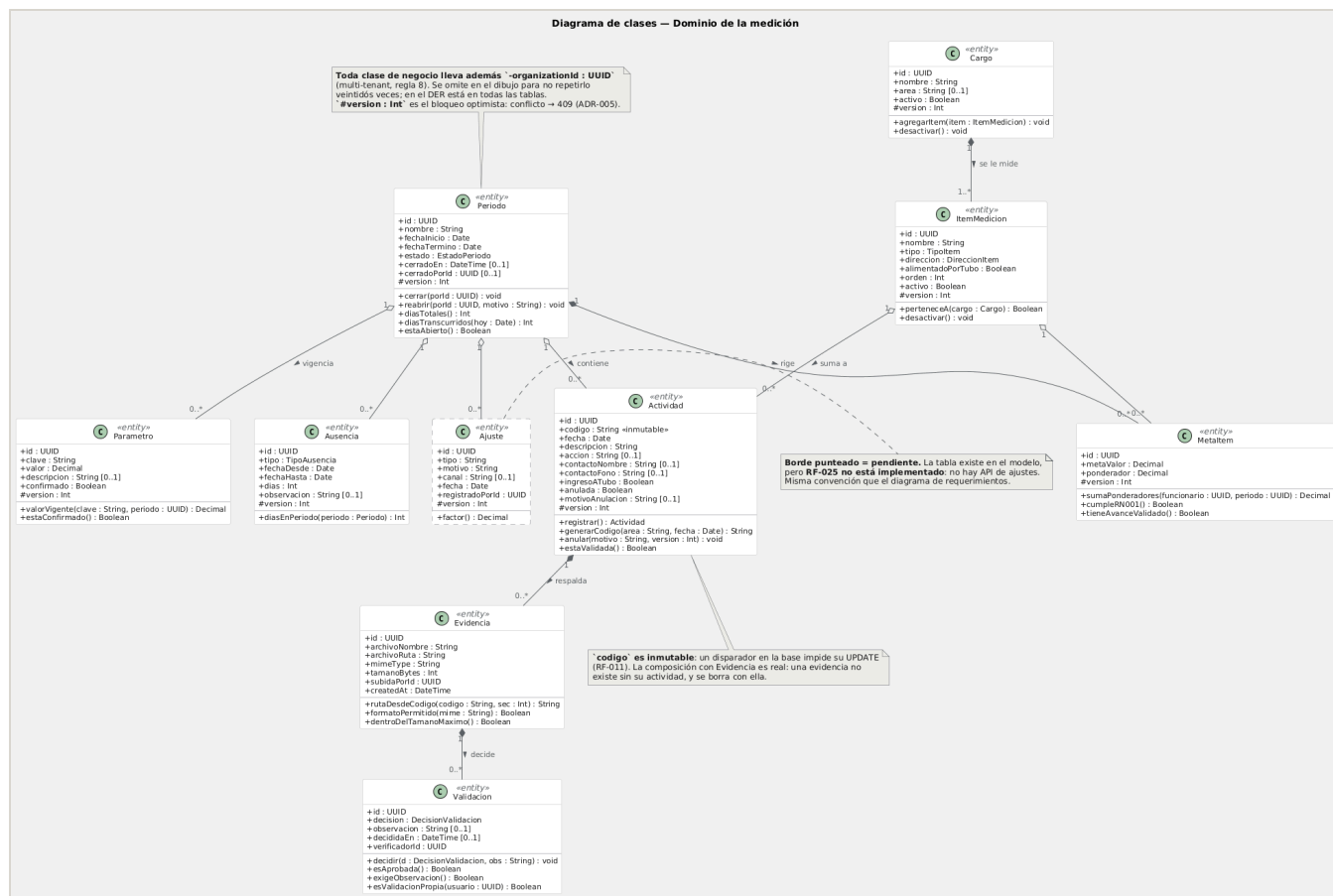


Panorama por capas

Fuente: [pum1/23-clases-panorama.puml](#)

Las invariantes que no se negocian viven en la base de datos, no en el código: el código de la actividad es inmutable por disparador, la bitácora tiene revocados `UPDATE` y `DELETE`, y hay restricciones `CHECK` de RUT, de fechas y de metas. Una regla que solo vive en la aplicación se salta con un script.

5. Dominio de la medición



Dominio de la medición

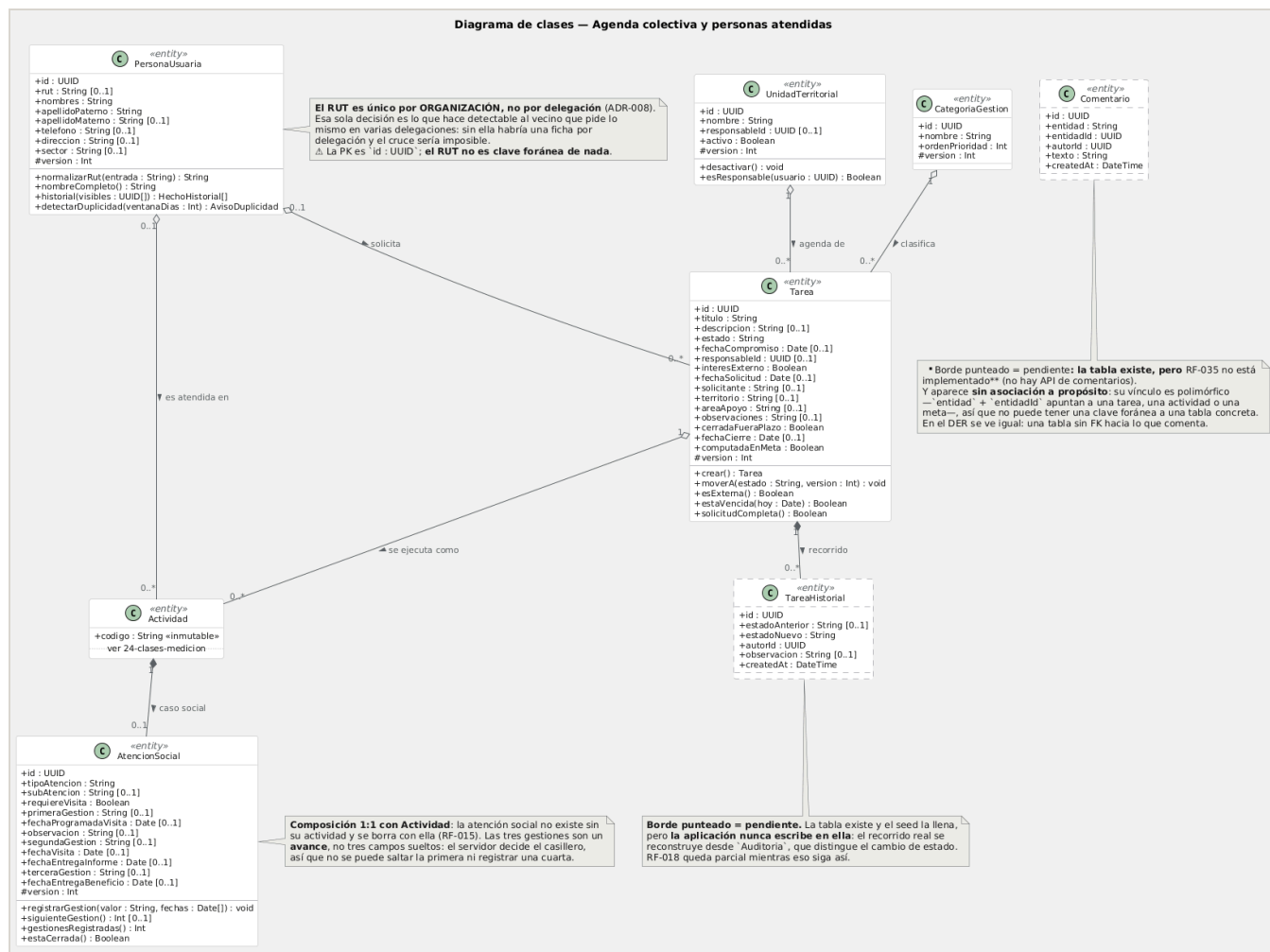
Fuente: [puml/24-clases-medicion.puml](#)

Es la cadena que va del **cargo** al **puntaje**: un cargo tiene sus ítems medibles, cada funcionario recibe una meta y un ponderador por ítem y período, registra actividades, las respalda con evidencia, y **solo la evidencia aprobada suma**.

Dos cosas que conviene mirar:

- **Cargo *-- ItemMedicion es composición, no agregación.** Un ítem de medición no existe fuera de su cargo: se borra con él.
- **Periodo o-- Actividad es agregación.** La actividad pertenece a un período, pero tiene identidad propia y sobrevive al cierre: por eso el período cerrado **no se borra ni se toca** (RN-013).

6. Agenda colectiva y personas atendidas



Agenda colectiva y personas

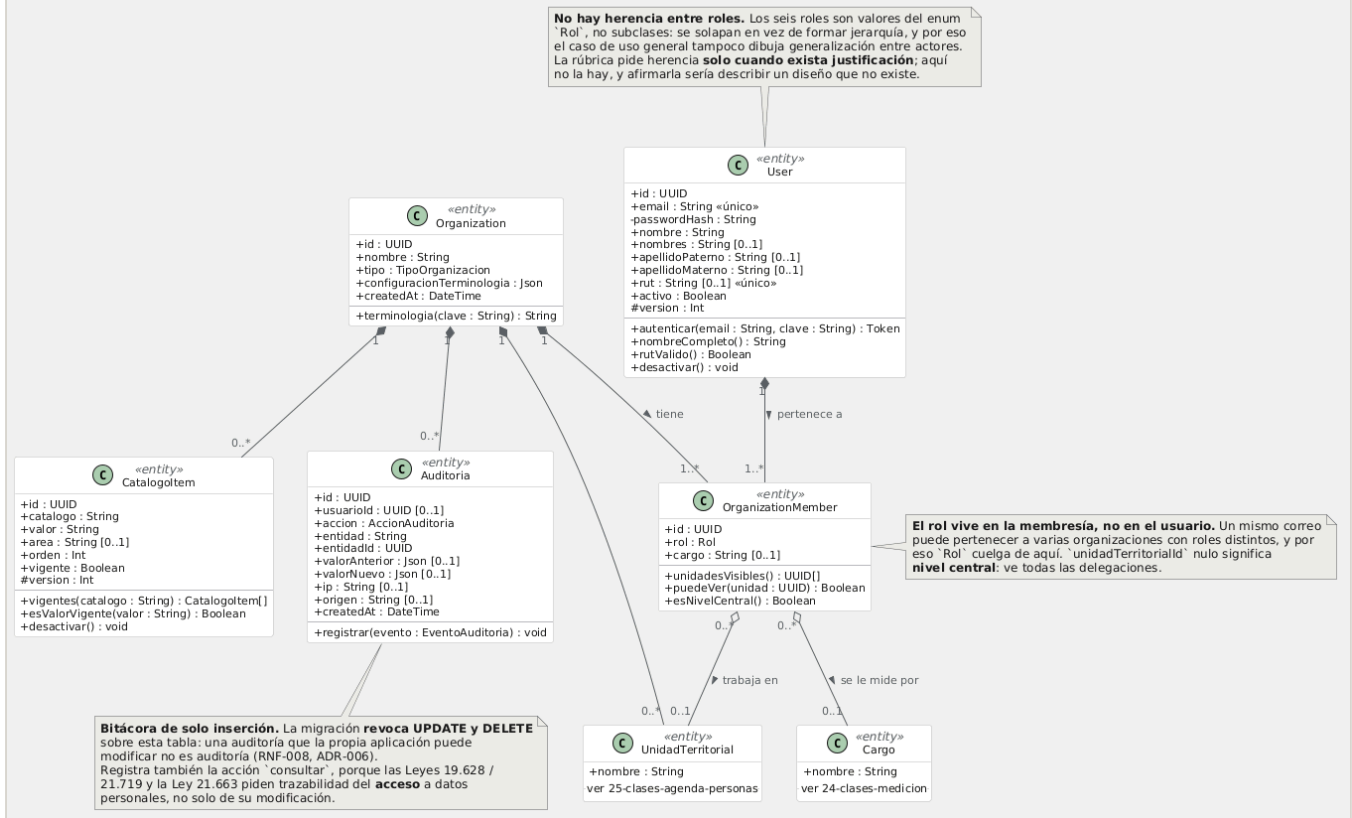
Fuente: [puml/25-clases-agenda-personas.puml](#)

Aquí está la decisión que da sentido a todo el control del cliente: **PersonaUsuaría es única por organización, no por delegación**. Una sola ficha por persona en todo el municipio es lo que permite que el sistema note que el mismo vecino pidió lo mismo en cinco delegaciones. Con una ficha por delegación, el cruce sería imposible por construcción.

Comentario aparece **sin asociación a propósito**: su vínculo es polimórfico (**entidad** + **entidadId** apuntan a una tarea, una actividad o una meta), así que no puede tener clave foránea a una tabla concreta. En el DER se ve igual.

7. Plataforma, identidad y trazabilidad

Diagrama de clases — Plataforma, identidad y trazabilidad



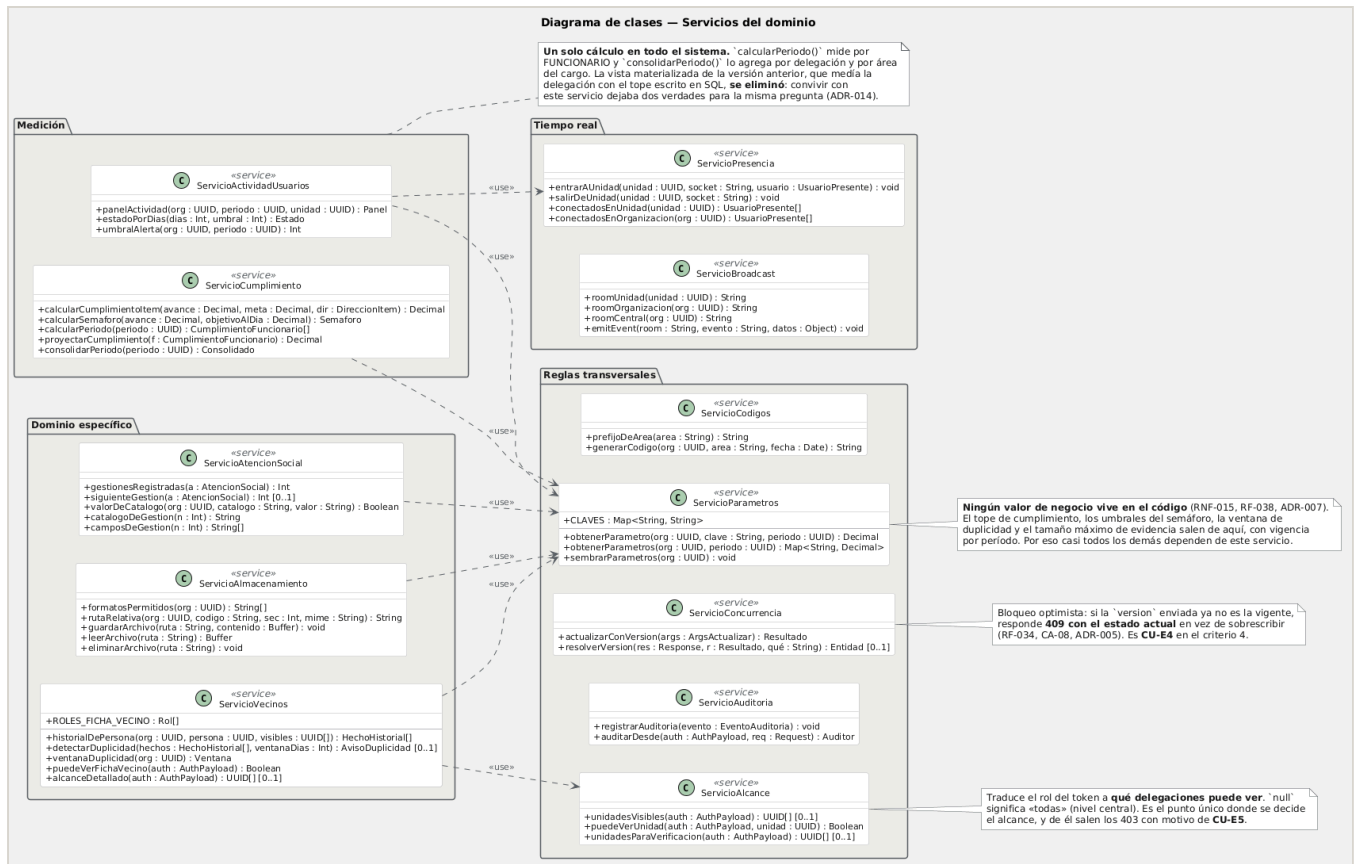
Plataforma y trazabilidad

Fuente: [puml/26-clases-plataforma.puml](#)

El rol vive en la membresía, no en el usuario. Un mismo correo puede pertenecer a varias organizaciones con roles distintos, y `unidadTerritorialId` nulo significa nivel central: ve todas las delegaciones.

Auditoria es una tabla de solo inserción. La migración revoca `UPDATE` y `DELETE` sobre ella: una auditoría que la propia aplicación puede modificar no es auditoría. Registra también la acción `consultar`, porque las Leyes 19.628 / 21.719 y la Ley 21.663 piden trazabilidad del **acceso** a datos personales, no solo de su modificación.

8. Los servicios



Servicios del dominio

Fuente: [puml/27-clases-servicios.puml](#)

Aquí están las operaciones. El dominio de este sistema es persistente y deliberadamente delgado: la lógica es transversal a varias entidades —el cumplimiento cruza meta, actividad, evidencia, validación y ausencia— y meterla dentro de una de ellas la escondería.

Casi todos los servicios dependen de `ServicioParametros`, y eso es una regla del proyecto hecha dibujo: **ningún valor de negocio vive en el código**. El tope de cumplimiento, los umbrales del semáforo, la ventana de duplicidad y el tamaño máximo de evidencia salen de la tabla de parámetros, con vigencia por periodo.

Por qué no hay herencia

La rúbrica pide herencia, composición o agregación **solamente cuando exista justificación en el diseño**. En SGR no la hay:

- **Los seis roles no son subclases.** Son valores de un enumerado, y se **solapan** en vez de formar jerarquía: el Verificador valida pero no ve el tubo; el Coordinador y el Administrador comparten el panel de actividad sin que uno herede del otro. Es la misma razón por la que el [caso de uso general §3.2](#) no dibuja generalización entre actores, y las dos decisiones tienen que ser la misma o el conjunto se contradice.
- **Las entidades no comparten ancestro.** Todas llevan `organizationId` y casi todas `version`, pero eso es una **convención transversal**, no una superclase: no hay comportamiento heredado que justifique inventarla.

Afirmar una herencia que el código no tiene sería exactamente el tipo de incoherencia que la rúbrica castiga.

9. Los tipos, y su equivalencia con MySQL

Los diagramas usan tipos UML neutros. El DER y el script usan los de MySQL, y esta es la tabla que los une — es la que hay que poder mostrar si preguntan por qué el diagrama dice `UUID` y el script dice `CHAR(36)` :

Tipo en el diagrama	PostgreSQL (implementación real)	MySQL (entregable)
<code>UUID</code>	<code>uuid</code>	<code>CHAR(36)</code>
<code>String</code>	<code>text</code>	<code>VARCHAR(n)</code>
<code>String</code> (texto largo)	<code>text</code>	<code>TEXT</code>
<code>Boolean</code>	<code>boolean</code>	<code>TINYINT(1)</code>
<code>Int</code>	<code>integer</code>	<code>INT</code>
<code>Decimal</code>	<code>numeric(p,s)</code>	<code>DECIMAL(p,s)</code>
<code>Date</code>	<code>date</code>	<code>DATE</code>
<code>DateTime</code>	<code>timestampz</code>	<code>DATETIME(3)</code>
<code>Json</code>	<code>jsonb</code>	<code>JSON</code>
Enumerados	<code>CREATE TYPE ... AS ENUM</code>	<code>ENUM(...)</code>

Fechas y horas no son lo mismo (ADR-002): un día calendario va en `Date` y un instante en `DateTime` . Por eso `fecha` de una actividad es `Date` y `createdAt` es `DateTime` .

10. Coherencia: clase ↔ tabla ↔ caso de uso

Es la tabla que la rúbrica usa para comprobar el criterio transversal. Se lee en cualquier dirección.

Clase	Tabla del DER	Casos de uso que la usan	Estado
<code>Organization</code>	<code>organizations</code>	Todos (multi-tenant)	✓
<code>User</code>	<code>users</code>	Todos	✓
<code>OrganizationMember</code>	<code>organization_members</code>	Todos (alcance por rol)	✓
<code>UnidadTerritorial</code>	<code>unidades_territoriales</code>	CU-06, CU-07, CU-09, CU-12	✓
<code>CategoriaGestion</code>	<code>categorias_gestion</code>	CU-06	✓
<code>Tarea</code>	<code>tareas</code>	CU-01, CU-06, CU-07	✓
<code>TareaHistorial</code>	<code>tarea_historial</code>	—	✗ Sin escritura
<code>Periodo</code>	<code>periodos</code>	CU-01, CU-04, CU-05, CU-09, CU-12	✓

Clase	Tabla del DER	Casos de uso que la usan	Estado
Parametro	parametros	CU-02, CU-05, CU-09, CU-11, CU-12	✓
Cargo	cargos	CU-01, CU-04	✓
ItemMedicion	items_medicion	CU-01, CU-04, CU-05	✓
MetaItem	metas_item	CU-04, CU-05, CU-09	✓
PersonaUsuaría	personas_usuarias	CU-01, CU-06, CU-11	✓
Actividad	actividades	CU-01, CU-02, CU-08, CU-10, CU-11	✓
Evidencia	evidencias	CU-02, CU-03	✓
Validacion	validaciones	CU-03, CU-05, CU-09	✓
AtencionSocial	atenciones_sociales	CU-08, CU-11	✓
Ausencia	ausencias	CU-05, CU-09	🟡 Sin API propia
Ajuste	ajustes	—	🟡 RF-025 pendiente
CatalogoItem	catalogo_items	CU-02, CU-06, CU-08	🟡 Solo lectura
Comentario	comentarios	—	🟡 RF-035 pendiente
Auditoria	auditoria	CU-01 a CU-04, CU-06 a CU-08, CU-10 a CU-12	✓

22 clases, 22 tablas. No hay clase sin tabla ni tabla sin clase, y el verificador lo comprueba.

⚠ **Tres clases no tienen caso de uso, y no es un olvido.** Ajuste (RF-025) y Comentario (RF-035) están declarados fuera de esta iteración; TareaHistorial tiene tabla pero **la aplicación nunca escribe en ella**, así que RF-018 queda parcial. Los tres van con borde punteado. Es la misma honestidad que el diagrama de requerimientos: el modelo dice qué necesita el sistema, no qué alcanzamos a construir.

11. Cómo se comprueba y cómo se regenera

```
cd frontend
npm run verificar:entrega          # coherencia entre todos los artefactos
npm run puml -- ../docs/entrega/puml --png # regenera los PNG desde los .puml
```

El verificador comprueba, entre otras cosas, que **toda clase dibujada exista como modelo en schema.prisma** y que **todo modelo del esquema esté dibujado**. Es la comprobación que evita el error más caro de este criterio: un diagrama de clases que describe un sistema que no es el nuestro.